웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

React 와 Express 로 구현한 JWT 인증 게시판의 구현 내용과 확인 결과를 정리한 보고서입니다.

01 서비스 소개와 구현 범위

학번 202212238 이름: 정성수 제출일: 2026-10-27

서비스 이름 JWT 인증 게시판

기술 조합과 버전 React / Express / node.js v24.22.1, React 19.3.0, Express 5.3.0

GitHub 저장소 URL https://github.com/wjdtjdtn222-coder/JWT-Assignment

소스 코드 기준 main /

서비스 소개

이 서비스는 게시글을 읽고 로그인 후 글을 작성하는 사용자를 위한 간단한 게시판이다. React 로 화면을 만들고 Express 로 인증 및 게시글 API 를 구현했다. SQLite 를 사용하여 게시글을 저장한다. 제공 템플릿을 기반으로 JWT 발급·검증, 인증 헤더 전송, 작성자 권한 확인, 만료 시 로그아웃 처리를 보완했다.

아래 완료 여부는 실제로 확인한 범위를 기준으로 작성했다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend-express/src/app.js; frontend-react/src/main.jsx	게시글 조회 화면
로그인과 JWT 발급	완료	src/auth.js; src/app.js	Alice 로그인 성공
인증과 내 정보 조회	완료	src/auth.js; src/app.js	/api/me 정상 및 401 확인
본인 글 작성·수정·삭제	완료	src/auth.js; src/app.js; frontend-react/src/main.jsx	Alice 작성·수정, Bob 403
만료·로그아웃·오류 표시	완료	frontend-react/src/api.js	만료 시 로그아웃 표시
DB 데이터 유지	부분	backend-express/src/store.js	SQLite 사용, 재시작 유지 미검증

웹서비스설계 02

구조와 변경 내역

브라우저, API 서버, 데이터베이스의 역할과 인증 흐름을 정리했다.

요청 흐름

브라우저(React, 5173) → Express API(8000) → SQLite 순서로 요청이 전달된다. 로그인 시 브라우저가 이메일·비밀번호를 POST /api/auth/login 으로 보내고 서버가 계정을 확인해 JWT 를 발급한다. 브라우저는 로그인 세션에 accessToken 과 사용자 정보를 보관하며, 보호된 API 요청에 Authorization: Bearer <토큰>을 붙인다. 게시글 수정 시 서버가 먼저 토큰을 검증하고 토큰의 사용자 ID 와 게시글 authorId 를 비교한다. 다르면 수정하지 않고 403 을 반환한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
src/auth.js / issueToken·currentUserId·assertOwner	인증 관련 보조 코드	JWT 발급·검증 및 작성자 비교	보호 API 의 인증·권한 적용
frontend-react/src/api.js / login·request	API 호출	로그인 세션, Bearer 헤더, 401 초기화	토큰 전송·만료 처리
frontend-react/src/api.js / canEdit	게시판 UI	작성자에게만 수정·삭제 표시	사용자 경험 개선

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-express/src/auth.js , issueToken(user)

코드 발췌

```
export function issueToken(user) {
  const secret = process.env.JWT_SECRET;
  const ttl = Number(process.env.TOKEN_TTL || 90);

  if (!secret||!/^[0-9a-fA-F]{64}$/.test(secret)){
    throw new Error("JWT_SECRET 설정 오류");
  }
  if (!Number.isSafeInteger(ttl)||ttl<=0){
    throw new Error("TOKEN_TTL 설정 오류");
  }
  return jwt.sign(
    {sub:String(user.id)},Buffer.from(secret,"hex"),{
      algorithm: "HS256",
      expiresIn: ttl,
      issuer: process.env.JWT_ISSUER,
      audience: process.env.JWT_AUDIENCE
    }
  );
}
```

동작과 응답 코드 설명

로그인 요청이 들어오면 서버는 이메일과 비밀번호의 입력값을 검사한 뒤, `verifyCredentials()`를 통해 계정 정보가 일치하는지 확인한다. 계정이 일치하면 `issueToken()`을 호출하여 JWT 를 발급한다.

JWT 에는 사용자 ID 를 나타내는 `sub`, 발급자를 나타내는 `iss`, 사용 대상을 나타내는 `aud`, 발급 시간인 `iat`, 만료 시간인 `exp` 가 포함된다. 서명 알고리즘은 HS256 을 사용하며, 토큰 유효 시간은 환경변수 `TOKEN_TTL` 로 설정한다. 기본값은 90 초이다.

로그인에 성공하면 HTTP **200** 과 함께 `accessToken`, `expiresIn`, `user` 를 JSON 으로 반환한다. 정상 응답에는 별도의 오류 `code` 가 없다. 이메일 또는 비밀번호가 일치하지 않으면 HTTP **401** 과 JSON 오류 코드 `INVALID_CREDENTIALS` 를 반환한다. JWT 비밀 키나 유효 시간 설정이 잘못된 경우에는 토큰 발급 과정에서 오류가 발생하며, 서버의 오류 처리 방식에 따라 응답한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js (login, authorizationHeaders)

코드 발췌

```
function authorizationHeaders() {
  if (session.accessToken){
    return {
      Authorization: `Bearer ${session.accessToken}`
    };
  }
  // TODO FE-02: 메모리에 보관한 토큰을 Bearer 헤더로 보내세요.
  return {};
}
export async function login(email, password) {
  const data = await request("/auth/login",{
    method:"POST",
    body: JSON.stringify({email,password}),
  });
  session={accessToken: data.accessToken, user:data.user,};
  return data.user;
```

동작과 응답 코드 설명

로그인 요청이 성공하면 서버에서 받은 accessToken 과 사용자 정보를 프론트엔드의 session 변수에 저장한다. 인증이 필요한 API 를 호출할 때는 authorizationHeaders() 함수가 저장된 토큰을 가져와 Authorization: Bearer 토큰값 형식으로 요청 헤더에 추가한다.

로그인 성공 시 서버는 HTTP 200 과 함께 토큰과 사용자 정보를 반환하며, 정상 응답에는 별도의 오류 code 가 없다. 인증 정보가 잘못된 경우에는 HTTP 401 과 INVALID_CREDENTIALS 를 반환한다. 또한 인증 요청에서 HTTP 401 이 발생하면 request() 함수가 clearSession()을 호출하여 로그인 상태를 해제한다.

구현 과정에서 Authorization 헤더의 문자열 표기 오류로 보호 API 가 HTTP 401 을 반환하는 문제가 있었다. 이를 수정한 후 게시글 작성이 정상적으로 동작하는 것을 확인했으며, 토큰 만료 시 로그인 상태가 해제되는 것도 확인했다.

웹서비스설계 03 계속

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-express/src/auth.js (currentUserId)

코드 발췌

```
export function currentUserId(req) {
  const authorization = req.headers.authorization;

  if (!authorization || !/^Bearer \S+$/.test(authorization)) {
    throw new ApiError(401, "TOKEN_REQUIRED", "토큰이 필요합니다.");
  }

  const token = authorization.split(" ")[1];

  try {
    const decoded = jwt.verify(
      token,
      Buffer.from(process.env.JWT_SECRET, "hex"),
      {
        algorithms: ["HS256"],
        issuer: process.env.JWT_ISSUER,
        audience: process.env.JWT_AUDIENCE
      }
    );
```

동작과 응답 코드 설명

인증이 필요한 API 요청이 들어오면 currentUserId() 함수가 Authorization 헤더에서 Bearer 토큰을 확인한다. 토큰이 없거나 헤더 형식이 올바르지 않으면 HTTP 401 과 TOKEN_REQUIRED 를 반환한다.

토큰이 존재하면 jwt.verify()를 이용하여 HS256 알고리즘으로 서명을 검증하고, 만료 시간과 발급자(iss), 사용 대상(aud)을 확인한다. 이후 exp 와 sub 클레임의 형식을 검사하고, sub 에 저장된 사용자 ID 가 실제 등록된 사용자인지 확인한다. 모든 검증에 성공하면 사용자 ID 를 반환하여 보호된 API 에서 사용한다.

검증에 실패한 토큰은 HTTP 401 과 INVALID_TOKEN 을 반환하고, 만료된 토큰은 HTTP 401 과 TOKEN_EXPIRED 를 반환한다. 실제 테스트에서도 정상 토큰으로 /api/me 요청에 성공했고, 토큰이 없거나 변조되거나 만료된 경우에는 HTTP 401 응답을 확인했다.

3 4 작성자 권한 검사

파일 / 함수: backend-express/src/auth.js (assertOwner)

코드 발췌

```
export function assertOwner(userId, authorId) {
  if(userId!==authorId){
    throw new ApiError(403,"FORBIDDEN","작성자만 수정하거나 삭제할 수 있습니다.");
  }
```

동작과 응답 코드 설명

게시글 수정 또는 삭제 요청이 들어오면 서버는 JWT 검증을 통해 로그인한 사용자의 ID 를 확인한다. 이후 assertOwner() 함수에서 로그인한 사용자 ID 와 게시글에 저장된 작성자 ID 를 비교하여 권한을 검사한다.

두 ID 가 일치하면 작성자로 판단하여 수정 또는 삭제 작업을 진행한다. 반대로 일치하지 않으면 HTTP 403 Forbidden 응답을 반환하여 다른 사용자의 게시글을 변경하지 못하도록 차단한다.

실제 테스트에서는 Alice 계정으로 게시글을 작성하고 수정하는 데 성공했다. 이후 Bob 계정으로 Alice 의 게시글을 수정하거나 삭제하도록 요청했을 때 HTTP 403 응답이 발생하는 것을 확인했다. 프론트엔드에서도 canEdit() 함수를 사용하여 본인 게시글에만 수정·삭제 버튼이 표시되도록 구현했다.

테스트 결과

아래 표는 실제로 확인한 테스트만 PASS 로 적고, 확인되지 않은 테스트는 미실행으로 구분했다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없이 목록 및 존재하는 글 상세조회	200, 정상 공개 조회	200, 정상 공개 조회	pass	밑에 전부 evidence 에
T02	Alice 계정으로 로그인	200, JWT 와 공개 사용자 정보, 화면 로그인 표시	200, JWT 와 공개 사용자 정보, 화면 로그인 표시	pass	추가 기록 필요
T03	잘못된 비밀번호 및 없는 이메일로 각각 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	401 INVALID_CREDENTIALS, 토큰 미발급	pass	추가 기록 필요
T04	토큰 없이 /me 와 글 등록 요청	각각 401 TOKEN_REQUIRED	각각 401 TOKEN_REQUIRED	pass	추가 기록 필요
T05	Alice 토큰으로 글 작성 후 /me 조회	201, authorId 1, /me 는 Alice	201, authorId 1, /me 는 Alice	pass	추가 기록 필요
T06	Alice 토큰으로 본인 글 수정, 삭제	200 및 204, 결과가 DB 에 반영	200 및 204, 결과가 DB 에 반영	pass	추가 기록 필요
T07	ALice 글을 Bob 의 정상 토큰으로 수정	403 FORBIDDEN, 원문 유지	403 FORBIDDEN, 원문 유지	pass	추가 기록 필요
T08	Alice 글을 bob 의 정상 토큰으로 삭제	403 FORBIDDEN, 글 유지	403 FORBIDDEN, 글 유지	pass	추가 기록 필요
T09	유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청	401 INVALID_TOKEN, 데이터 변경 없음	401 INVALID_TOKEN, 데이터 변경 없음	pass	추가 기록 필요

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T10	짧은 TTL 로 발급하고 실제 만료 뒤 /me 및 변경요청	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	pass	추가 기록 필요
T11	로그인 후 로그아웃, 다음 /me 요청	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	pass	추가 기록 필요
T12	bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가	400 BAD_INPUT, 위조 글 미생	400 BAD_INPUT, 위조 글 미생성	pass	추가 기록 필요
T13	본인 토큰으로 빈 제목, 공백 본문, 길이초과 입력을 각각 전송	400 BAD_INPUT, DB 변경 없음	400 BAD_INPUT, DB 변경 없음	pass	추가 기록 필요
T14	없는 양의 정수 ID 의 조회, 수정, 삭제 및 잘못도니 ID 조회	없는 글 404, 잘못된 ID 400; 수정·삭제에는 정상 토큰 사용	없는 글 404, 잘못된 ID 400; 수정·삭제에는 정상 토큰 사용	pass	추가 기록 필요
T15	글을 만든 뒤 서버 종료, 재실행	기존 글과 작성자·내용 유지	기존 글과 작성자·내용 유지	pass	추가 기록 필요
T16	로그인 후 페이지 새로고침	공개 글 유지, 메모리 로그인 상태 해제	공개 글 유지, 메모리 로그인 상태 해제	pass	추가 기록 필요
T17	토큰 없는 수정, 삭제 요청과 bob 계정의 정상 작성, 본인 수정, 삭제	무토큰은 401, Bob 본인 작업은 성공	무토큰은 401, Bob 본인 작업은 성공	pass	추가 기록 필요

웹서비스설계

AI 활용과 참고 자료

ChatGPT 를 구현 설명, 코드 수정 및 테스트 점검에 사용했다.

AI 사용 여부: 사용 (ChatGPT, GPT-6)

AI 활용 목적

본 과제에서는 JWT 기반 사용자 인증과 게시글 접근 권한을 구현하는 과정에서 생성형 AI 를 보조도구로 활용하였다.

AI 는 JWT 인증방식에 대한 개념 이해 및 코드 제안, API 테스트 절차에 활용, 오류 원인 분석 및 테스트 결과 정리, 보고서 작성, README 작성에 활용하였다.

AI 활용 내용

1. JWT 의 발급 및 검증 과정과 Authorization: Bearer 헤더를 통한 인증방식을 이해하는데 AI 를 활용하였다.
2. 과제 흐름에 따른 코드 제안을 요구하였다.
3. AI 의 도움을 받아 브라우저 개발자도구의 console 과 network 기능을 활용하여 api 응답을 확인하였다.
4. 테스트과정에서 오류가 났을 때 오류 원인 분석과 대응에 활용하였다.
5. 보고서 작성에 AI 의 도움을 받았다.

AI 활용과정에서의 본인 역할

ai 가 제시한 설명과 테스트 코드를 참고하되 실제 프로그램의 실행과 테스트는 직접 수행하였다. 브라우저 개발자 도구를 통해 HTTP 상태코드와 오류 응답을 확인하였고 게시글 작성, 수정, 삭제 및 로그인 상태변화도 직접 점검하였다. AI 의 답변을 그대로 결과로 인정하지 않고 실제 실행결과를 기준으로 기능의 정상 동작 여부를 판단하고자 하였다.

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

사용자가 authorId 를 다른 사람의 ID 로 바꿔 타인의 이름으로 글을 작성할 수 있다. 서버는 클라이언트가 주장한 작성자가 아니라 검증된 JWT 의 사용자 ID 를 기준으로 작성자와 권한을 판단해야 한다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼을 숨기는 것은 화면상의 제한일 뿐 직접 API 를 호출하면 우회할 수 있다. 서버의 assertOwner(userId, authorId)가 403 FORBIDDEN 을 발생시켜 실제 수정을 차단한다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰 내용을 읽을 뿐 서명이나 유효성을 검증하지 않는다. verify 는 서명과 만료 등을 검증한다. 이 구현은 HS256, issuer, audience 및 만료 조건을 검사하도록 구성했다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 브라우저 세션의 토큰을 지우고, 만료는 서버에서 토큰을 더 이상 인정하지 않는 것이다. 새로고침 후 로그인 상태 유지 여부는 세션 저장 방식에 따라 달라진다. 이미 복사된 유효 토큰은 단순 로그아웃만으로 서버에서 자동 폐기되지 않는다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

Authorization 헤더에 따옴표를 잘못 사용해 토큰이 전송되지 않았고 401 이 발생했다. 템플릿 문자열로 수정한 뒤 글 작성 성공을 확인했다. canEdit 의 오타로 흰 화면이 나왔을 때도 오타를 고쳐 화면이 정상 동작하는지 확인했다.